6D Object Pose Estimation: Accurate RGB-D Multi-modal Optimization and Refinement

Elziario Paduano Marco Pavanati Sadaf Qureshi Klejsi Zeqaj
Politecnico di Torino
Corso Duca degli Abruzzi, 24, 10129 Torino, Italy
{elziario.paduano, marco.pavanati, sadaf.qureshi, klejsi.zeqaj}@polito.it
https://github.com/ElzPad/6D-Pose-Estimation

Abstract

Estimating the 6-Degree-of-Freedom (6-DoF) pose of objects is a fundamental challenge in computer vision, essential for robotic manipulation and augmented reality applications. This task becomes particularly demanding when dealing with cluttered scenes.
In this work, we enhance an RGB-only 6D pose estimation pipeline by incorporating depth cues and geometric refinement. We propose ARMOR (Accurate RGB-D Multi-modal Optimization and Refinement), an RGB-D extension that introduces a dedicated depth feature extractor and performs mid-level multimodal fusion to improve both rotation and translation prediction. To enforce 3D consistency beyond the learned estimates, we further apply a point-to-point Iterative Closest Point (ICP) refinement step that aligns the object CAD model with a depth-derived point cloud, initialized from the network output. We evaluate the goodness of the proposed extensions on the LINEMOD benchmark using ADD and ADD-S with an appropriate acceptance threshold, and report quantitative and qualitative results that isolate the impact of depth fusion and ICP refinement.

1. Introduction

6D object pose estimation involves determining the precise 3D translation vector (x, y, z) and the 3D rotation matrix (R) of an object relative to a camera coordinate system. This capability is a cornerstone for autonomous systems, enabling robots to interact with their environment through grasping and manipulation, and facilitating realistic object rendering in augmented reality.
Unlike 2D object detection task, that has been revolutionized by the advent of Convolutional Neural Networks (CNNs), retrieving the full 6D pose remains challenging due to the variety of scenes we can have (e.g. different lighting conditions, objects with symmetries and so on).

Figure 1. Example scene from the LINEMOD dataset. This benchmark contains texture-less objects in cluttered environments, presenting significant challenges for resolving geometric ambiguities in 6D pose estimation.

Standard benchmarks like the LINEMOD dataset [2], Fig 1, specifically test these capabilities on texture-less objects, where purely visual features (RGB) may be insufficient for resolving geometric ambiguities. Traditional approaches often involve complex processing pipelines, which can increase computational cost and reduce robustness in noisy environments.

Deep Learning methods, instead, typically decouple the problem into detecting the object and regressing its pose. However, most of the existing architectures, ignore the rich geometric informations provided by depth sensors, and solely rely on RGB input. While RGB-D sensors are now very frequent, effectively fusing these two distinct modalities, color texture and geometric depth, remains an open research problem. Standard fusion techniques, such as element-wise summation or concatenation, often fail to account for the ”semantic gap” and inconsistent scales between RGB and depth features.

In this work, we propose *ARMOR*, a modular, two-stage pipeline that addresses these limitations through

systematic integration of depth information and geometric refinement. First, we employ an object detector to localize objects of interest. Second, we introduce an RGB-D pose estimation network with a dedicated depth feature extractor that captures geometric cues—surface orientations, object silhouettes, and range distributions—from the depth channel. These depth features are fused with RGB representations from ResNet-50 backbones via mid-level concatenation to have an initial estimation of the rotation and translation. Finally, we apply an Iterative Closest Point (ICP) refinement stage that aligns the 3D CAD model with the depth-derived point cloud, initialized from the network predictions, to achieve high-precision final 6D poses.

In summary, the contribution of our work is:

RGB-D Feature Integration Pipeline: We enhance RGB-only pose estimation by integrating depth information through a dedicated DepthCNN feature extractor and mid-level feature fusion. This approach combines the semantic richness of RGB data with the metric precision of depth data, improving rotation and translation prediction for texture-less objects.

Geometric Refinement via ICP: We incorporate an Iterative Closest Point refinement stage leveraging depth-based geometric constraints to refine neural predictions, avoiding local minima through learned initialization.

Systematic Evaluation and Ablation: We provide comprehensive analysis on the LINEMOD benchmark using ADD and ADD-S metrics, isolating the contributions of depth fusion and ICP refinement. Our results demonstrate that RGB-D integration significantly outperforms RGB-only baselines, validating the effectiveness of combining learned features with geometric constraints for 6D pose estimation.

2. Related Work

Deep Learning for 6D Pose Estimation The field of 6D object pose estimation has transformed with deep learning, evolving from traditional template matching to end-to-end CNN-based architectures handling texture-less objects in cluttered scenes [7]. Early approaches relied exclusively on RGB input, leveraging convolutional networks for pose regression. However, RGB-only methods face limitations estimating absolute 3D translation and resolving geometric ambiguities for texture-less objects. Widespread RGB-D sensor availability in robotic manipulation has motivated multimodal fusion strategies integrating color with geometric depth, though RGB and depth modalities operate at different semantic levels, requiring careful architectural design.

RGB-D Fusion Approaches Recent RGB-D fusion research has explored several paradigms for pose estimation. PoseCNN combines learned RGB features with geometric ICP refinement but treats depth as optional post-processing. DenseFusion performs pixel-wise fusion by associating 3D point features with RGB pixels, achieving high accuracy at computational cost. PointFusion demonstrates effective global feature concatenation from independent encoders without complex attention mechanisms.PointFusion's efficient concatenation strategy, integrate depth features early in the pipeline rather than as post-processing, and employ ICP refinement with improved initialization from RGB-D fusion.

2.1. PoseCNN

Xiang et al. introduced PoseCNN: A Convolutional Neural Network for 6D Object Pose Estimation in Cluttered Scenes [5], a seminal CNN-based approach for 6D object pose estimation in cluttered scenes. The fully convolutional architecture simultaneously performs object segmentation, 3D translation estimation, and rotation regression from RGB images. Translation prediction localizes the object's center in the image plane and estimates camera distance, transforming 2D observations into 3D coordinates using camera intrinsics, while rotation is represented as a quaternion regressed from RGB features. PoseCNN's defining characteristic is its two-stage pipeline: the CNN provides initial pose estimates from visual features, subsequently refined through geometric alignment with depth data. This hybrid approach—combining learned visual representations with classical geometric optimization—demonstrated significant improvements on LINEMOD, establishing an influential paradigm.

2.2. PointFusion

Xu et al. introduced PointFusion: Deep Sensor Fusion for 3D Bounding Box Estimation [6] for 3D object detection, demonstrating that heterogeneous sensor fusion through simple concatenation can be highly effective. The architecture processes RGB images through CNN and raw point clouds through PointNet independently before combining outputs. PointFusion presents two fusion strategies: global fusion that concatenates high-level feature vectors for direct 3D bounding box regression, and dense fusion combining point-wise with global features for spatially-aware predictions. Experiments on autonomous driving benchmarks showed that properly designed concatenation-based fusion achieves competitive performance without complex attention mechanisms, demonstrating that simple concatenation at appropriate feature levels effectively bridges the semantic gap between RGB and geometric modalities.

2.3. DenseFusion

Wang et al. proposed DenseFusion [4], a heterogeneous architecture achieving state-of-the-art 6D object pose estimation through pixel-wise RGB-D feature fusion. The architecture processes RGB images through a fully convolutional network and depth point clouds through PointNet, maintaining separate encoders for complementary information extraction. DenseFusion's key innovation is its dense fusion mechanism: for each 3D point, geometric features from PointNet are associated with corresponding RGB pixel features by projecting points onto the image plane using camera intrinsics. These paired features are concatenated and processed to produce pixel-wise dense embeddings capturing appearance and geometry. The network outputs per-pixel pose predictions aggregated through confidence-weighted voting, with an end-to-end learned iterative refinement module progressively correcting errors without external geometric algorithms.

3. Method

3.1. RGB-Only Baseline Architecture

The baseline system implements a three-stage modular pipeline designed to estimate the full 6-DoF pose of a target object—defined by a rotation matrix $\mathbf{R} \in SO(3)$ and a translation vector $\mathbf{t} \in \mathbb{R}^3$—from a single RGB image. The architecture decouples the task into object 2D localization, orientation regression, and spatial translation estimation. Each stage is trained independently, allowing specialized optimization of the detection, rotation, and translation components before they are integrated into the final inference pipeline. The latter operates strictly on RGB data and geometric metadata, such as object diameter, without utilizing depth information, which is excluded from both the training and inference processes.

3.2. Data Preprocessing and Input Representations

The data pipeline is designed to transform raw imagery into task-specific representations for orientation and spatial estimation. A defining difference between the two stages is the spatial extent of the input.

Local Representation for Rotation For orientation regression, the system extracts local RGB patches centered on the target object. Using the 2D bounding box provided by the detection layer, the object is cropped with a padding factor to provide sufficient local context. These crops are resized to 224×224 pixels to match ResNet-50 input dimension, focusing the network's capacity on the internal textures and edge gradients necessary to resolve 3D orientation.

Global Representation for Translation Conversely, the translation module utilizes full-frame imagery also resized to ResNet-50 backbone without cropping. This global view allows the model to infer the object's position and distance by observing its relationship with the scene layout, camera viewpoint, and relative scale within the image plane.

3.2.1. Augmentation and Normalization

During the training phase, both translation and rotation pipelines apply a suite of photometric augmentations to improve robustness against environmental variations. For the rotation stage, augmentations are strictly non-geometric (e.g., color jitter, Gaussian blur, and noise) to ensure that the visual cues for orientation remain consistent with the ground-truth labels. The translation stage adopts a similar photometric suite. For both stages, the final input tensors are normalized using standard ImageNet statistics (mean and standard deviation). In the translation stage, ground-truth coordinates are converted from millimeters to meters to ensure numerical stability during the regression of spatial coordinates.

Regarding the object detection phase, the transformation strategy involves random rotations, spatial translations, and scaling adjustments as well as color jittering to make the model robust to variations in lighting conditions and sensor noise.

3.3. 2D Object Detection

The first stage employs a YOLO11s-based detector to localize the target object within the image plane. Given an input image, the detector predicts a set of bounding boxes, class labels, and confidence scores. The detector is initialized from the COCO-pretrained YOLO model to take advantage of robust low-level feature representations before being specialized in the target domain. The pipeline selects the highest-confidence bounding box corresponding to the target class, which serves as the spatial constraint for subsequent 3D estimation modules.

3.4. Rotation Estimation via Quaternion Regression

Orientation is estimated through a ResNet-50-based regressor that predicts a unit quaternion representing the object's rotation. The input to this module is a cropped RGB patch of the object, extracted using the detected 2D bounding box. The model utilizes a ResNet-50 backbone, pretrained on ImageNet, as a feature extractor. A one-hot encoded vector representing the object identity is then concatenated with the visual feature vector. This mid-fused representation is processed by a multi-layer perceptron (MLP) to output a 4D vector, representing the rotation quaternions, which is explicitly L_2-normalized to ensure it resides on the unit 3-sphere. The training objective utilizes a geodesic loss to minimize the angular distance between the predicted and ground-truth orientations. Given a predicted unit quaternion

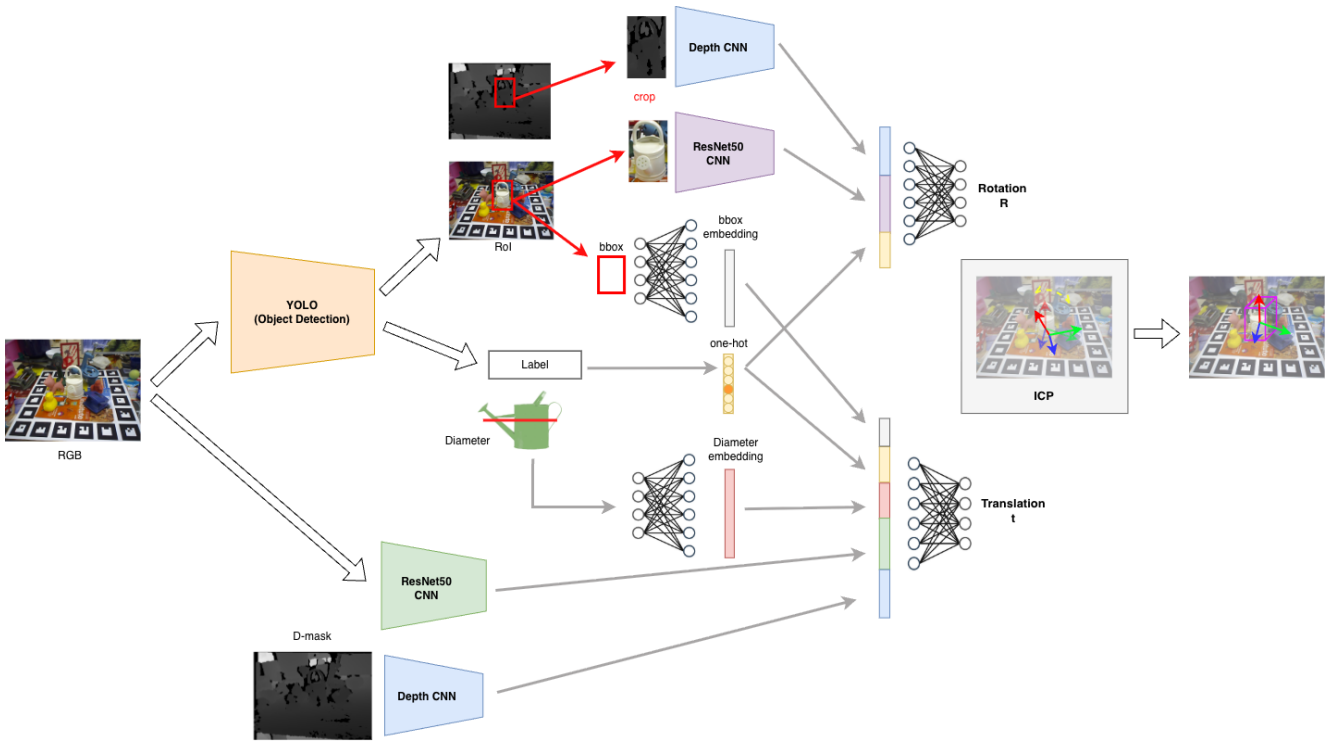

Figure 2. ARMOR architecture: Initial 6D pose is estimated via multi-modal fusion of RGB, DepthCNN features, and geometric embeddings, then refined through classical ICP to achieve final metric precision.

$\hat{\mathbf{q}}$ and a ground-truth unit quaternion $\mathbf{q}$, the loss is formulated as follows

$$L_{rot} = 2 \arccos(|\langle \hat{\mathbf{q}}, \mathbf{q} \rangle|) \tag{1}$$

where the absolute value of the inner product accounts for the double-cover property of quaternions, ensuring that $\mathbf{q}$ and $-\mathbf{q}$ represent the same rotation.

3.5. Translation Estimation and Feature Fusion

While the 2D object detection stage effectively determines the object's lateral position within the image plane, recovering the complete 3D translation vector $\mathbf{t} = [x, y, z]^\top$ from monocular RGB data remains a significant challenge. Traditional methods, such as those proposed by Kehl et al. [3] and Xiang et al. [5], utilize known camera intrinsics and geometric projections to infer spatial coordinates; however, resolving the inherent depth ambiguity and scale variations through purely deterministic geometric means is often non-trivial.

To address these difficulties, the proposed baseline incorporates a dedicated translation regression module designed to directly predict the object's origin in the camera coordinate system. In contrast to the rotation module, which operates on local patches, this stage employs a global-to-local feature fusion strategy to capitalize on broader scene context:

- **Global Visual Features:** A secondary ResNet-50 backbone, also pretrained on ImageNet, processes the **full RGB image** to capture contextual information regarding the camera viewpoint and scene scale. This helps the model to understand the position of the detected object with respect to the entire scene.
- **Geometric Embeddings:** The normalized 2D bounding box coordinates and the physical object diameter—available in the object metadata and retrieved by the YOLO predicted label—are processed through dedicated MLPs to capture non-linear relationships between projected 2D size and 3D depth.
- **Object Conditioning:** A one-hot object label encoding is incorporated to allow the network to learn object-specific scale and depth priors.

These heterogeneous features are concatenated into a unified representation and passed through a final regression head for the estimation of the three translation values. The loss function is defined as a weighted mean squared error (MSE), where errors in the z-component (depth) are more heavily penalized to reflect the inherent difficulty and importance of accurate depth estimation in RGB-only systems. The translation loss is defined as:

$$L_{trans} = \sum_{j \in x,y,z} w_j (\hat{t}_j - t_j)^2 \tag{2}$$

where the weight vector $\mathbf{w}$ is set to fix values, with higher penalty to the depth component to encourage more precise spatial positioning.

3.6. Taking advantage of the depth

The primary contribution to the baseline is the systematic integration of depth information to resolve the geometric ambiguities inherent in monocular RGB-only systems. The enhanced pipeline, that we call *ARMOR*, adopts a two-stage approach: first, it leverages a multi-modal feature fusion strategy to estimate an initial 6D pose; second, it employs a geometric iterative refinement stage to achieve a high-precision final result. To support this, a dedicated depth processing stream is introduced, utilizing a compact convolutional encoder, designated as *DepthCNN*, to extract local geometric cues—such as surface orientations and object silhouettes—directly from the depth channel. Fig. 2 shows the whole ARMOR architecture.

In the first stage, these depth-derived features are fused with the visual representations from the RGB backbones to provide a robust initial orientation and translation estimate. This multi-modal approach ensures that the network benefits from both the rich semantic textures of RGB data and the precise metric constraints of depth data.

In the second stage, the initial pose is further improved through an Iterative Closest Point (ICP) refinement process. This stage aligns the 3D CAD model of the object with the observed depth point cloud, iteratively minimizing the alignment error to produce the final 6D pose estimate. By initializing the ICP algorithm with the learned predictor's initial estimate, the system effectively avoids local minima and achieves superior accuracy.

3.6.1. Depth Feature Extraction

The *DepthCNN* processes single-channel depth inputs normalized to metric units. The architecture consists of four successive convolutional blocks, each employing a 3×3 kernel, batch normalization, and ReLU activation, to progressively downsample the spatial resolution from 224×224 to 14×14. A final convolutional layer followed by adaptive average pooling results in a fixed 256-dimensional geometric feature vector that captures surface orientations, object silhouettes, and range distributions.

As this encoder is integrated into both the translation and rotation stages, it is optimized end-to-end within each respective module alongside the ResNet-50 backbone and MLP heads. Following the modular strategy established for the RGB-only baseline, this approach allows the *DepthCNN* to develop specialized weights tailored to the specific inputs of each task: for orientation regression, the encoder learns to extract local geometric features from cropped depth patches, while for translation estimation, it learns to interpret the global metric scale and scene context from the full-frame depth map.

3.6.2. RGB-D Pose Prediction

As previously outlined, the pose prediction modules are modified to integrate depth features alongside the previously established inputs.

- **Rotation:** The *ResNetRotationRGBD* module extends the baseline orientation head by concatenating RGB features with the depth ones obtained from the RGB-D patch. The final fused representation, which also includes the one-hot object labeling, is regressed to an L_2-normalized unit quaternion to which corrisponds the initial $\mathbf{R}_{init}$.
- **Translation:** Similarly, the *ResNetTranslationRGBD* module enhances the baseline translation stream by incorporating depth embedding into the feature mid-fusion stage. The resulting joint representation—comprising visual features (RGB and depth), the one-hot object identity, and geometric embeddings (normalized bounding box and object diameter)—is used to regress the initial 3D translation $\mathbf{t}_{init}$.

3.7. Geometric Pose Refinement via ICP

Following the initial pose estimation $(\mathbf{R}_{init}, \mathbf{t}_{init})$, the system performs a geometric refinement inspired by Xiang et al. [5], utilizing a point-to-point Iterative Closest Point (ICP) [1] algorithm to align the 3D model with the observed depth measurements. This refinement stage is implemented using the Open3D library [8], which provides a robust and efficient ICP solver.

3.7.1. Point Cloud Reconstruction

Using the camera intrinsics, the depth map is back-projected to form an observed point cloud $\mathcal{P}_{obs}$. For a pixel (u, v) with depth z, the 3D point $\mathbf{p} = [x, y, z]^\top$ in the camera coordinate system is computed as:

$$x = \frac{(u - c_x)z}{f_x}, \quad y = \frac{(v - c_y)z}{f_y}, \quad z = z \tag{3}$$

where (f_x, f_y) and (c_x, c_y) represent the focal lengths and principal point, respectively. To reduce noise and background interference, reconstruction is restricted to pixels within the predicted 2D bounding box. Only valid depth values are considered, and a minimum of 50 points is required to ensure sufficient geometric constraints for refinement.

3.7.2. Iterative Alignment

The CAD model vertices $\mathcal{P}_{obj}$ are transformed into the camera frame using the initial pose estimate, forming the source point cloud $\mathcal{P}_{src}$. ICP then iteratively estimates a corrective rigid transformation $T_{refine} = [\mathbf{R}_{refine}|\mathbf{t}_{refine}]$ by minimizing the point-to-point alignment error:

$$E(\mathbf{R}_{refine}, \mathbf{t}_{refine}) = \sum_{i=1}^{N} \|\mathbf{p}_i - (\mathbf{R}_{refine}\mathbf{q}_i + \mathbf{t}_{refine})\|^2 \tag{4}$$

where $\mathbf{p}_i \in \mathcal{P}_{obs}$ is the closest observed point to $\mathbf{q}_i$ within a fixed correspondence distance threshold.

We employ a point-to-point ICP variant with a maximum of 70 iterations and a correspondence threshold of 5.0 mm, which balances robustness to sensor noise with convergence stability. Convergence is determined based on relative changes in the estimated transformation parameters between successive iterations.

3.7.3. Convergence Assessment and Pose Composition

The alignment quality is assessed using the ICP fitness metric, defined as the fraction of source points that find valid correspondences within the distance threshold. Refinement is accepted only if the fitness exceeds 0.3, ensuring that the update is supported by sufficient geometric evidence.

The final pose is obtained by composing the corrective transformation with the initial estimate:

$$\mathbf{R}_{final} = \mathbf{R}_{refine}\mathbf{R}_{init} \tag{5}$$

$$\mathbf{t}_{final} = \mathbf{R}_{refine}\mathbf{t}_{init} + \mathbf{t}_{refine} \tag{6}$$

4. Experiments

4.1. Dataset

LINEMOD Dataset To evaluate our proposed method, we utilize the LINEMOD dataset, a standard benchmark for 6D object pose estimation. The dataset comprises 13 texture-less objects, each with approximately 1,200 images captured in heavily cluttered scenes under varying lighting conditions. Each frame provides synchronized RGB and depth images at a resolution of 640×480 pixels, together with pixel-wise segmentation masks. Ground-truth annotations include 2D bounding boxes, corresponding 3D CAD models (in `.ply` format), and the 6D pose parameterized by a 3×3 rotation matrix and a 3×1 translation vector. The camera intrinsic parameters are fixed across the dataset, with focal lengths $f_x \approx 572$ and $f_y \approx 574$. In addition, the dataset contains symmetric objects (e.g., *eggbox* and *glue*), which require symmetry-aware evaluation protocols when assessing orientation accuracy.

4.2. Metrics

We evaluate 6D pose accuracy using the Average Distance of Model Points (ADD) and its symmetry-aware variant ADD-S. Given a set of 3D model points $\mathcal{M}$, a predicted pose $(\hat{\mathbf{R}}, \hat{\mathbf{t}})$, and ground-truth pose $(\mathbf{R}, \mathbf{t})$, ADD measures the mean Euclidean distance between corresponding model points 'projected' by the two poses:

$$\text{ADD} = \frac{1}{|\mathcal{M}|} \sum_{x \in \mathcal{M}} \|(\mathbf{R}x + \mathbf{t}) - (\hat{\mathbf{R}}x + \hat{\mathbf{t}})\| \tag{7}$$

For symmethric objects (e.g. eggbox, glue), where multiple rotations map the object to the same visual appearance, the ADD metric is ambigous. We instead use ADD-S, which computes the mean distance to the *nearest neighbor* point in the model.

$$\text{ADD-S} = \frac{1}{|\mathcal{M}|} \sum_{x_1 \in \mathcal{M}} \min_{x_2 \in \mathcal{M}} \|(\mathbf{R}x_1 + \mathbf{t}) - (\hat{\mathbf{R}}x_2 + \hat{\mathbf{t}})\| \tag{8}$$

To summarize pose quality in a robust and comparable manner across objects of different scales, we report the fraction of test instances for which the estimated pose achieves ADD $< 0.1 \cdot d$, with d being the CAD model diameter.

4.3. Implementation Details

We implemented the full pipeline in Python using PyTorch (torch/torchvision) for the pose networks, Ultralytics for YOLO-based detection, and Open3D for geometric refinement, together with standard utilities for mesh handling and image processing (e.g. trimesh, NumPy).

The LINEMOD data were converted into a YOLO-compatible layout and split deterministically into 80/20 train/validation partitions per object using seeded randomization for reproducibility. The detector (YOLOv11s) was fine-tuned for 50 epochs (batch size 16) using an initial learning rate of 5×10^{-4} and standard augmentations. The RGB baselines and RGB-D translation network utilized a two-phase schedule: initially training task heads with frozen backbones for 30–40 epochs (lr 10^{-3}, batch 64), followed by fine-tuning the full network (lr 10^{-4}, batch 32) for a total of 80–100 epochs (80 for the RGB rotation and 100 for both translation networks). In contrast, the proposed RGB-D rotation network was trained fully unfrozen for 110 epochs (lr 5×10^{-4}, batch 32). Training was governed by a dynamic learning rate scheduling strategy to ensure stable convergence. We employed the *ReduceLROnPlateau* scheduler: the learning rate was decayed by a factor of **0.5** whenever the loss failed to improve for 3 consecutive epochs ($patience = 3$). This adaptive reduction allowed the model to fine-tune weights effectively during the later stages of training.

For the RGB-D extension, raw depth (mm) is converted to meters, clipped at 3 m, and normalized to [0,1] (invalid pixels set to 0); depth crops are additionally masked by the segmentation and extracted around the predicted box with 20% padding, before resizing to 224×224. Finally, for ICP refinement we set a distance threshold in mm of 5.0 for ICP correspondence and a maximum number of ICP iterations equal to 70.

4.4. Results

Quantitative Comparison Table 1 compares the effects of our two extensions, depth fusion and ICP refinement, under the ADD(-S)@0.1d criterion, with other previous work. The proposed pipeline demonstrates a cumulative improvement across stages, culminating in a mean accuracy of

Table 1. Per-object pose accuracy on LINEMOD using ADD(-S) @ $0.1d$. Symmetric objects are in **bold**. Left: our methods. Right: prior works (PoseCNN, PointFusion, DenseFusion). Best per row is highlighted in bold.

Object	RGB Baseline	RGB-D	ARMOR	PoseCNN	PointFusion	DenseFusion (iter.)
ape	25.8	29.4	85.1	77.0	70.4	**92.3**
benchvise	95.5	95.1	95.5	**97.5**	80.7	93.2
camera	73.4	85.1	93.4	93.5	60.8	**94.4**
can	87.5	87.9	95.0	**96.5**	61.1	93.1
cat	71.2	77.5	95.3	82.1	79.1	**96.5**
driller	93.7	**95.8**	**95.8**	95.0	47.3	87.0
duck	42.6	51.0	**94.4**	77.7	63.0	92.3
eggbox	**100.0**	**100.0**	**100.0**	97.1	99.9	99.8
glue	**100.0**	**100.0**	**100.0**	99.4	99.3	**100.0**
holepuncher	75.4	79.4	91.5	52.8	71.8	**92.1**
iron	94.8	93.1	93.1	**98.3**	83.2	97.0
lamp	96.3	95.5	96.3	**97.5**	62.3	95.3
phone	90.4	91.6	**95.2**	87.7	78.8	92.8
Mean	80.4	83.0	**94.7**	88.6	73.7	94.3

94.7% for the full RGB-D + ICP method. In comparison to other approaches, ARMOR significantly outperforms PointFusion (73.7%) and PoseCNN (88.6%). Finally, ARMOR, performs comparably with respect to the iterative refinment version of DenseFusion (94.3%), with a negligible difference of 0.4%, validating the efficacy of the proposed architecture.

Symmetry and Object Analysis We observe a clear gap in performance between symmetric and asymmetric objects. ARMOR demonstrates exceptional robustness on symmetric classes, achieving 100% for both the *eggbox* and *glue*. This indicates that the ADD-S metric handles geometric ambiguities during training. Differently, asymmetric objects with limited texture, such as the *ape* and *duck*, present significant challenges for the networks, yielding low initial RGB baselines of 25.8% and 42.6%, respectively. However, larger and geometrically distinct objects like the *benchvise* and *driller* maintain high accuracy ($> 93\%$) even without depth information, suggesting that RGB features are sufficient for objects with unique silhouettes.

Ablation Sudy: Depth and Refinment The integration of depth information and geometric refinement contributes unequally to the final performance. The fusion of depth features (RGB to RGB-D) yields a modest mean improvement of 2.6%, primarly resolving scale ambiguity for specific objects like the *camera* (+11.7%). In contrast, geometric refinement via ICP brings the most significant gain (+11.7%), giving significant improvement for small, texture-less objects like the *ape* (29.4% $\rightarrow$ 85.1%). A key advantage of ICP is that it requires no additional learning. However, ICP functions as a local optimizer and therefore depends on the quality of the initial pose estimate: refinement is applied only to hypotheses whose alignment fitness exceeds a predefined threshold, and incorrect initial predictions cannot be recovered. Furthermore, we observe a critical trade-off between accuracy and latency: while increasing maximum iterations from 10 to 70 boosts accuracy from 87.4% (+4.4% with respect to RGB-D) to 94.7% (+11.7%), the computational cost becomes prohibitive for real-time tasks. Consequently, we find that a moderate budget (30–50 iterations) provides the optimal balance for robotic manipulation, securing high precision without sacrificing the low-latency requirements of the system.

5. Conclusions

We presented ARMOR, a robust framework for estimating 6D poses of known objects from RGB-D images. Our approach integrates dense color and depth information to construct a unified feature representation, and leverages the Iterative Closest Point (ICP) algorithm to rigorously refine the geometric alignment of the predictions.
By augmenting deep feature extraction with iterative depth-based refinement, our method achieves higher accuracy compared to RGB-only baselines and demonstrates increased stability in handling texture-less objects. Furthermore, our experiments validate that incorporating geometric constraints significantly enhances pose estimation performance, providing a reliable foundation for robotic manipulation tasks in cluttered environments.

References

[1] P.J. Besl and Neil D. McKay. A method for registration of 3-d shapes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 14(2):239–256, 1992. 5

[2] Stefan Hinterstoisser, Vincent Lepetit, Slobodan Ilic, Ste-

fan Holzer, Gary Bradski, Kurt Konolige, and Nassir Navab. Model based training, detection and pose estimation of texture-less 3d objects in heavily cluttered scenes. 2012. 1

[3] Wadim Kehl, Fabian Manhardt, Federico Tombari, Slobodan Ilic, and Nassir Navab. SSD-6D: making rgb-based 3d detection and 6d pose estimation great again. *CoRR*, abs/1711.10006, 2017. 4

[4] Chen Wang, Danfei Xu, Yuke Zhu, Roberto Martín-Martín, Cewu Lu, Li Fei-Fei, and Silvio Savarese. Densefusion: 6D object pose estimation by iterative dense fusion. *CoRR*, abs/1901.04780, 2019. 3

[5] Yu Xiang, Tanner Schmidt, Venkatraman Narayanan, and Dieter Fox. PoseCNN: A convolutional neural network for 6D object pose estimation in cluttered scenes. *CoRR*, abs/1711.00199, 2017. 2, 4, 5

[6] Danfei Xu, Dragomir Anguelov, and Ashesh Jain. Pointfusion: Deep sensor fusion for 3D bounding box estimation. *CoRR*, abs/1711.10871, 2018. 2

[7] Chen Yuanwei, Mohd Hairi Mohd Zaman, and Mohd Faisal Ibrahim. A review on six degrees of freedom (6d) pose estimation for robotic applications. *IEEE Access*, 12:161002–161017, 2024. 2

[8] Qian-Yi Zhou, Jaesik Park, and Vladlen Koltun. Open3d: A modern library for 3d data processing. *CoRR*, abs/1801.09847, 2018. 5